COMP719 – OPTIMIZATION AND MODELLING

SEMESTER PROJECT

PROJECT TITLE: THE CAMPUS LAB SPACE OPTIMIZER

FOCUS: DISCRETE OPTIMIZATION, RESOURCE ALLOCATION, AND INTRODUCTORY HEURISTICS

1. PROJECT SCENARIO

The University has **four computer labs** of varying sizes and software configurations. There are **ten different modules** that require access to a computer lab at some point during the semester. Each module has: *a fixed number of enrolled students*, and *a specific software requirement*.

The department is currently facing two issues:

- **Overcrowding** – assigning a module to a lab that is too small for its class size.
- **Underutilization** – assigning small classes to very large labs, leaving many seats empty.

Important Assumption (Scheduling)

Assume that **all modules are scheduled at different times**. Therefore, **a lab may host multiple modules over the semester**, but **each module must be assigned to exactly one lab**.

Your Goal

Develop a **Python-based optimization tool** that assigns each module to one compatible lab such that:

- Every student has a seat, and
- The **total number of unused seats** across all module–lab assignments is minimized.

2. THE DATASET

The Labs

Lab ID	Lab Name	Capacity	Software Installed
L1	Advanced Lab A	100	Python, Java, MATLAB
L2	Coding Lab B	60	Python, Java
L3	Data Lab C	40	Python, MATLAB
L4	Legacy Lab D	30	Java, MATLAB

The Modules

Module Code	Module Name	Students	Software Needed
M1	AI Systems	85	Python
M2	Control Systems	45	MATLAB
M3	Object Oriented	25	Java
M4	Machine Learning	55	Python
M5	Numerical Methods	35	MATLAB
M6	Software Design	15	Java
M7	Enterprise Systems	90	Java
M8	Scripting Basics	30	Python
M9	Signal Processing	20	MATLAB
M10	Data Structures	50	Python

3. Constraints and Objective

Decision Variables:

Let:

$\mathbf{x_{ij} = 1}$ if module *i* is assigned to lab *j,* and **0** otherwise.

Constraints:

Unique Assignment: Each module must be assigned to exactly one lab:

$\Sigma_j x_{ij} = 1$ for all modules *i*

Capacity Constraint: A module can only be assigned to a lab if the lab capacity is sufficient:

If $\text{Students}_i > \text{Capacity}_j$, then $x_{ij} = 0$

Software Compatibility: A module can only be assigned to a lab that has the required software installed.

Objective Function:

For each valid module–lab assignment, unused seats are calculated as:

Unused Seats = Lab Capacity − Number of Students

The objective is to:

Minimize the total unused seats across all assigned modules.

4. EXPECTED APPROACH (BY PHASE)

This project is intentionally structured in stages to gradually build your solution.

Phase 1: Modelling and Setup (30%)

- Clearly define decision variables, constraints, and the objective function.
- Represent the data in Python (e.g., dictionaries or tables).
- Implement feasibility checks for capacity and software compatibility.
- Deliverables:
    - Presentation
    - Short proposal. *Use the provided proposal template to prepare the report*
        - Mathematical model (written clearly)
        - Initial Python code (data loading + structure)

Phase 2: Greedy Heuristic (20%)

- Implement a **simple greedy algorithm** to generate a feasible solution.
- Examples of acceptable greedy ideas include:
    - Assign larger classes first to the smallest compatible lab.
    - Assign modules in descending order of class size.
    - Any other clearly justified greedy rule.
- Your solution must:
    - Respect all constraints
    - Output a valid assignment and the total unused seats
- Deliverables:

- A partially working Python Program
- A Phase 2 Report. *Use the provided Phase 2 Report Template to prepare the report*
- Presentation by group members

Phase 3: Improvement Strategy (50%)

- Improve your greedy solution using a **basic local search** approach.
- Guidelines:
  - Define a simple neighbourhood (e.g., reassign one module to another compatible lab).
  - Accept changes that reduce total unused seats.
  - Advanced techniques are **not required**; a basic hill-climbing strategy is sufficient.
- Deliverables:
  - Final working Python program
  - A Final report describing results, challenges, and improvements (*Use the provided Final Report Template to prepare the report*).
  - Final presentation with a live demo

5. SUBMISSION SCHEDULE

Milestone	Deadline	Deliverables	Presentation
Group Finalisation	Feb 15	Final group list	N/A
Phase 1 (30%)	Feb 25	Proposal + Model + Initial Code	Feb 27
Phase 2 (20%)	Apr 8	Greedy implementation	Apr 10
Phase 3 (50%)	May 20	Final Code + Report	May 22

6. PRESENTATION RULES

1. First Page Must list all members and their **Percentage of Contribution** (e.g., 5 members = 20% each for equal work). 0% contribution results in 0 marks for that phase.
2. Different students must present at each phase.
3. While two members present, the Q&A session will target the **non-presenting** members to ensure total group participation.
4. Individual marks may vary from the group total based on contribution percentages and Q&A performance.

5. You may only make a single submission per group. As a group, nominate one member to make all submissions and ensure that everyone agrees on the final files. Double-check that all files work properly.
6. The submissions must contain the correct submission files for the project to be graded. Please follow each instruction carefully to avoid penalties.
7. You cannot change your group membership or split into subgroups. Submissions must be made as a group, based on the selections/allocations made on the Group Selection page. Any unauthorised changes to group composition will result in severe penalties, including disqualification from the assessment in some cases. Please collaborate closely with your group during this submission phase.
8. Late submissions are strongly discouraged. Submissions made after the due date (even by 1 minute) will automatically incur a penalty, with an additional 20% deduction for every overdue hour.
9. Please choose your group from the options available on Moodle. The group you choose will serve as both your project group and your tutorial group.
10. There are different groups available, and you can choose any group. While the groups are numbered, this does not necessarily indicate the order in which projects will be presented.
11. You will be able to view the members of each group once your classmates have assigned themselves to a group number. If you wish to form a group with specific classmates, you may approach them directly.
12. The group selection will close on Friday, February 15th, so please join a group before then.
13. Participation in a group is requirement, and you must be part of a group in order to submit your project. Failure to select a group on time will result in being placed in a random group and incurring a significant penalty to your class mark. Please move yourself out of the default groups and into a numbered group to avoid incurring any penalties smile. Do not stay in the default group!
14. Please choose your group from the options available on Moodle.

7. MARKING RUBRIC

Phase 1: Proposal & Initial Build (30%)

Criteria	Excellent	Satisfactory	Poor
Proposal (5%)	Clear objectives and mathematical formulation	Partial clarity	Incomplete
Initial Code (10%)	Correct structure and data handling	Buggy but present	Missing
Presentation (15%)	Clear explanations and confident answers	Basic understanding	Unable to explain

Phase 2: Mid-way Progress (30%)

Criteria	Excellent	Satisfactory	Poor
Implementation (10%)	All constraints correctly enforced	One constraint missing	Incorrect
Code Quality (5%)	Clean and readable	Understandable but messy	Hard to follow
Presentation (15%)	Clear explanations and confident answers	Basic understanding	Unable to explain

Phase 3: Final Delivery (40%)

Criteria	Weight	Description
Technical Approach	10%	Use of greedy + local search
Correctness	10%	Valid solution respecting all constraints
Final Report	10%	Clear discussion of results
Final Presentation	20%	Professional slides, demo, and Q&A

FINAL NOTE

This project prioritises clear modelling, correct constraint handling, and logical reasoning over advanced mathematics or complex algorithms. A simple, well-explained solution will score higher than an overly complicated one that is poorly justified.